AI Resume Screener: Constraint Satisfaction and Heuristic Search for Preliminary Candidate Screening

Nethra Chandragandhi T
Roll No. 2116240701357

Nethra G
Roll No. 2116240701358

Foundations of Artificial Intelligence – Project Paper, October 2026

***Abstract*—Manual screening of resumes is slow and can be inconsistent when many applications arrive for a single position. This paper presents AI Resume Screener, a web-based decision-support system built with Python and Flask that models preliminary screening as a constraint satisfaction problem (CSP). Mandatory requirements (education, minimum CGPA, work experience and required skills) are checked using CSP with constraint propagation; backtracking search confirms that a valid assignment exists; a heuristic evaluation assigns a suitability score out of 100; and best-first search orders the eligible candidates by that score. In a functional test with one sample resume and one set of job requirements, the candidate satisfied all four constraints, obtained a score of 90/100 and was placed first in the search order. The test shows that the pipeline works end to end, but it is not a statistical evaluation, and larger multi-resume experiments are identified as future work. The paper also discusses transparency, bias and regulatory considerations that apply to automated screening tools.**

***Index Terms*—resume screening, constraint satisfaction, backtracking search, heuristic evaluation, best-first search, decision support, recruitment.**

I. INTRODUCTION

Organisations that advertise on online job portals routinely receive far more resumes than a recruiter can read closely. Manual screening is time-consuming, and two recruiters may interpret the same requirement differently, which leads to inconsistent shortlists. Recent research therefore applies language models to screening: embedding models and large language models (LLMs) have been used to retrieve and rank resumes [1], to classify them into occupations [2] and to rate them against job criteria [3]. These approaches are powerful, but they also raise concerns. An audit of three embedding models found that they favoured White-associated names in 85.1% of tested cases [1], and a study of GPT-4 ratings found that LLM scores correlated only weakly with human ratings [3].

This paper takes a complementary route based on classical AI. Instead of learning a ranking from data, the screening decision is expressed as explicit constraints and an explicit scoring heuristic, so that every outcome can be traced to a rule that a recruiter can read and change. Such a system cannot understand paraphrased skills the way a language model can, but it does not require training data and it shows the reason for each decision.

The contributions of this work are as follows:

- A formulation of preliminary resume screening as a CSP over education, CGPA, experience and skills, followed by heuristic scoring and best-first ordering of the eligible candidates.
- A Python and Flask implementation that accepts job requirements and a resume, shows the extracted information, and reports constraint results, a suitability score and the search order.
- A functional test of the implementation, reported honestly as a single-case demonstration, together with a discussion of its limitations and of fairness and regulatory issues.

The rest of the paper is organised as follows. Section II reviews related work, Section III formulates the problem, Section IV describes the system, Section V reports the results, Section VI discusses implications and limitations, and Section VII concludes.

II. RELATED WORK

A. Learning-Based Resume Screening

Wilson and Caliskan simulated screening as a document-retrieval task using over 500 resumes and 500 job descriptions across nine occupations, and showed that the embedding models they tested systematically disadvantaged some demographic groups, with Black male names disadvantaged most [1]. Heakl et al. built a dataset of 13,389 resumes in 43 categories and fine-tuned BERT and Gemma models for resume classification [2]. Vaishampayan et al. compared zero-shot GPT-4 ratings with human ratings for 736 resumes and found that LLM scores were not interchangeable with human scores, although prompt design such as chain-of-thought improved them [3].

Industrial systems follow a similar direction. Li et al. use LLMs to extract fine-grained signals from job descriptions and report relative AUC gains of 1.70% for click-through and 5.97% for conversion in a talent-search ranker [4]. Jouanneau et al. distil an LLM teacher into a long-context re-ranker that produces skill-fit scores for person-job matching [5]. These systems rely on large amounts of data and compute, and their internal decisions are harder to inspect.

B. Information Extraction from Resumes

Converting a resume into structured fields is a prerequisite for any matching method. Kesim and Deliahmetoglu adapted six transformer models for named entity recognition on IT resumes, recognising entities such as degree, job title and skill [6]. In our system, extraction is intentionally kept simple, and the quality of later stages depends on it; Section VI discusses this limitation.

C. Classical Search and Constraint Satisfaction

Constraint satisfaction, backtracking search, heuristic evaluation and best-first search are standard techniques in AI problem solving [7]. A CSP is defined by variables, domains and constraints, and backtracking assigns variables one at a time and retreats when a constraint is violated. Best-first search expands the most promising node according to an evaluation function. To our knowledge these methods are less often applied to resume screening than learned models, which motivates the explicit and inspectable design explored here.

III. PROBLEM FORMULATION

A. Screening as a CSP

Let a job position define a set of mandatory requirements and let a candidate c be described by the attributes extracted from the resume. The screening problem is the triple $P = (X, D, C)$. The variables are X_1 (education), X_2 (CGPA), X_3 (experience) and X_4 (skills). Each variable X_i has a domain D_i of values that can be extracted from a resume. The constraints are:

- C_1: the candidate's qualification belongs to the set of accepted qualifications;
- C_2: CGPA is at least the minimum threshold;
- C_3: years of experience are at least the required minimum;
- C_4: the set of required skills is a subset of the candidate's skills.

A candidate is eligible when all constraints hold simultaneously:

$$E(c) = C_1 \wedge C_2 \wedge C_3 \wedge C_4 \quad (1)$$

Constraint propagation is used to prune early: if a mandatory constraint is violated, the candidate is marked ineligible without further search. Backtracking search then assigns the variables in order and confirms whether a complete valid assignment exists.

B. Heuristic Evaluation and Best-First Ordering

Eligible candidates are scored with a weighted heuristic

$$h(c) = \Sigma\, w_k \cdot m_k(c), \quad \Sigma\, w_k = 100 \quad (2)$$

where w_k is the number of points assigned to criterion k and $m_k(c) \in [0, 1]$ is the degree to which the candidate satisfies it. Best-first search keeps the eligible candidates in a priority queue ordered by $h(c)$ and removes them from highest to lowest score, so the removal order is the candidate ranking. The score is a measure of agreement with the configured criteria and must not be read as the probability that the candidate will perform well in the job.

IV. SYSTEM DESIGN AND IMPLEMENTATION

A. Architecture

The system is a web application written in Python with the Flask framework and developed in Visual Studio Code. Fig. 1 shows the processing pipeline. The recruiter enters the job requirements and uploads a resume. The system extracts the candidate's attributes, evaluates the constraints, searches for a valid assignment, computes the score and displays the result.

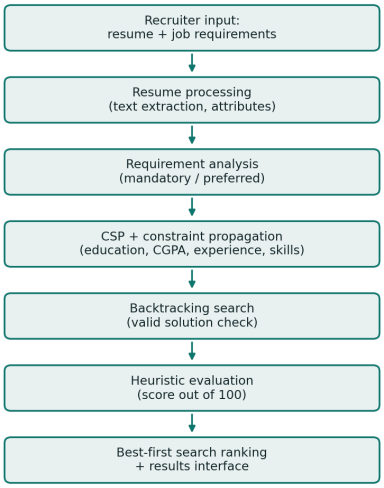


Fig. 1. Processing pipeline of AI Resume Screener.

B. Modules

Resume processing. Accepts the uploaded resume, extracts text and identifies education, CGPA, experience and skills. Unsupported or empty files should be rejected with a message, because errors at this stage propagate to every later stage.

Requirement analysis. Holds the job requirements entered by the recruiter: required skills, preferred skills, accepted qualifications, minimum CGPA and minimum experience.

Constraint evaluation. Applies the CSP with constraint propagation and reports PASS or FAIL for each mandatory constraint.

Search and scoring. Backtracking search confirms that a valid solution exists; heuristic evaluation then assigns points to each criterion, and best-first search orders the eligible candidates by score.

Results interface. Displays the extracted information, eligibility status, suitability score, constraint results and search results so that the recruiter can check the basis of each outcome.

C. Screening Procedure

The overall procedure is summarised below. For n candidates and k mandatory constraints, constraint checking takes $O(nk)$ time and ordering the eligible candidates takes $O(n \log n)$ time. This is an analytical estimate; running time was not measured in the present test.

Algorithm 1: Preliminary screening

```
Input: job requirements R, set of resumes S
for each resume s in S do
    c ← extract(s)                  // attributes
    if a mandatory constraint fails: mark ineligible; continue
    if BACKTRACK(c, R) fails: mark ineligible; continue
    c.score ← h(c)                  // Eq. (2)
    push c into priority queue Q keyed by c.score
rank ← 1
while Q is not empty do
    output (rank, pop highest-score candidate); rank ← rank + 1
```

V. EXPERIMENTS AND RESULTS

A. Experimental Setup

The application was run locally using Python and Flask. The test input consisted of one sample candidate resume and a

predefined set of job requirements covering education, minimum CGPA, work experience and required skills. The purpose of the test was functional: to verify that extraction, constraint evaluation, backtracking search, heuristic scoring and ordering work together and that the results are displayed correctly.

B. Results

One candidate was processed and one was found eligible. Table I lists the constraint results and Table II summarises the search and scoring results.

TABLE I
CONSTRAINT EVALUATION RESULT

Constraint	Status
Education	PASS
CGPA	PASS
Experience	PASS
Required skills	PASS

TABLE II
SEARCH AND SCORING SUMMARY

Measure	Value
Candidates processed	1
Candidates eligible	1
Backtracking: valid solution found	Yes
Heuristic suitability score	90 / 100
Best-first search position	1

The constraint evaluation reported PASS for all four mandatory constraints, and backtracking search confirmed that a valid solution exists. The heuristic evaluation gave the candidate 90 out of 100 points, and best-first search placed the candidate first in the search order. The interface displayed the extracted information, eligibility status, score, constraint results and search results.

C. Analysis

The results show that the pipeline operates correctly for the tested case and that each decision can be traced to a visible constraint result and a score. They do not establish accuracy, precision, recall or ranking quality. With a single resume, no ranking order between candidates can be observed, and no comparison with human screeners or with a learned model was made. The reported 90/100 is therefore an illustration of the scoring output and not a measure of system performance.

VI. DISCUSSION

A. Comparison with Learning-Based Screening

Table III compares the proposed rule-based design with embedding or LLM-based ranking in qualitative terms. The comparison reflects design properties and published findings [1], [3], not measurements made in this work.

TABLE III
QUALITATIVE COMPARISON

Criterion	LLM / embedding ranking	Proposed system
Transparency	Opaque scores	PASS/FAIL per constraint
Synonyms	Handled semantically	Weak; rule-based
Training data	Pre-trained model	None required
Bias evidence	Documented [1]	Untested; rule-dependent

B. Fairness, Transparency and Regulation

Explicit rules do not make a system automatically fair. A CGPA threshold or an accepted-qualification list can act as a proxy that disadvantages some groups, and the quality of the decision depends on correct extraction. Fairness checks on realistic data are therefore needed before any real use. Regulators already treat such tools with care. New York City Local Law 144 requires annual bias audits and candidate notice for automated employment decision tools, although compliance studies find that its scope is hard to verify [8]. The EU Artificial Intelligence Act lists AI systems used to recruit, filter applications and evaluate candidates among its high-risk systems [9]. The NIST AI Risk Management Framework gives general guidance on identifying and managing such risks [10]. For these reasons AI Resume Screener is designed as decision support: a recruiter reviews the original resumes and makes the final decision.

C. Limitations

- The test used one resume and one set of requirements, so no performance claim can be made.
- Extraction is simple and may miss information in unusual resume layouts or formats.
- Skill matching depends on the configured rules and does not recognise equivalent terms automatically.
- The heuristic weights are chosen by the developer and have not been validated against hiring outcomes.

VII. CONCLUSION AND FUTURE WORK

This paper presented AI Resume Screener, a decision-support system that models preliminary resume screening as a CSP, verifies it with backtracking search, scores eligible candidates with a heuristic and orders them with best-first search. A functional test confirmed that the pipeline runs end to end and produces understandable, traceable output, including a suitability score of 90/100 for the tested candidate.

Future work will (i) evaluate the system on a larger set of resumes and job descriptions and report extraction accuracy, ranking quality and processing time against human screeners; (ii) add named entity recognition [6] and semantic similarity so that related skills are recognised despite different wording; (iii) represent alternative skills with AND-OR graphs and AO* search; (iv) make the scoring weights configurable and explain each score; and (v) add fairness audits, secure storage and privacy protection in line with the guidance discussed above.

REFERENCES

[1] K. Wilson and A. Caliskan, "Gender, race, and intersectional bias in resume screening via language model retrieval," in *Proc. AAAI/ACM Conf. AI, Ethics, Soc. (AIES)*, vol. 7, no. 1, 2024, doi: 10.1609/aies.v7i1.31748.

[2] A. Heakl *et al.*, "ResuméAtlas: Revisiting resume classification with large-scale datasets and large language models," arXiv:2406.18125, 2024.

[3] S. Vaishampayan *et al.*, "Human and LLM-based resume matching: An observational study," in *Findings Assoc. Comput. Linguistics: NAACL 2025*, 2025.

[4] J. Li *et al.*, "Enhancing talent search ranking with role-aware expert mixtures and LLM-based fine-grained job descriptions," in *Proc. Conf. Empirical Methods Natural Lang. Process.: Ind. Track (EMNLP)*,

Suzhou, China, 2025, pp. 185–194, doi: 10.18653/v1/2025.emnlp-industry.13.

[5] W. Jouanneau, E. Jouffroy, and M. Palyart, “An efficient long-context ranking architecture with calibrated LLM distillation: Application to person-job fit,” arXiv:2601.10321, 2026.

[6] E. Kesim and A. Deliahmetoglu, “Named entity recognition in resumes,” arXiv:2306.13062, 2023.

[7] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[8] L. Wright *et al.*, “Null compliance: NYC Local Law 144 and the challenges of algorithm accountability,” in *Proc. ACM Conf. Fairness, Accountability, Transparency (FAccT)*, Rio de Janeiro, Brazil, 2024.

[9] European Parliament and Council of the EU, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),” *Off. J. Eur. Union*, Jul. 12, 2024.

[10] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.